

AN-NAJAH NATIONAL UNIVERSITY

Vehicle Rental Management System

AI Refactoring Report

Course	Software Engineering Project
Team Members	Mohammad abu salah - 12324714 Fadi Dallal - 12340833
Repository	https://github.com/najah-vehicle-rental/Vehicle-Rental-Management-System
Analysis Platform	SonarQube Cloud + GitHub Actions
Submission Date	1 August 2026

1. Purpose and Scope

This report records how AI assistance was used during refactoring and documentation. AI was used to identify small maintainability improvements, propose behavior-preserving changes, and help formulate verification steps. Every suggestion was reviewed before it was applied. Functional requirements and core implementation decisions remained the responsibility of the team.

2. Summary of AI-Assisted Work

File / Area	AI Suggestion	Decision
VehicleRepository.java	Extract duplicated literal into a constant	Accepted
ReminderServiceTest.java	Correct directory structure to match package	Accepted
RentalRepository.java	Replace System.out with java.util.logging.Logger	Accepted with tests
pom.xml / Javadocs	Generate documentation for all production classes and members	Accepted
Global package rename	Rename all uppercase package segments to lowercase	Rejected/deferred

3. Refactoring Record 1 - Duplicated Literal

File	src/main/java/najah/eng/Application/Persistence/VehicleRepository.java
SonarQube issue	Define a constant instead of duplicating the literal "Vehicles file not found." three times.
Prompt used	Refactor VehicleRepository to replace the repeated "Vehicles file not found." literal with a single constant. Preserve all current behavior and method return values.
Decision	Accepted

Original code

```
if (!Files.exists(filePath)) {
    System.out.println("Vehicles file not found.");
    return vehicles;
}
// Similar literal appeared in findById and updateStatus.
```

Refactored code

```
private static final String VEHICLES_FILE_NOT_FOUND =
    "Vehicles file not found.";

if (!Files.exists(filePath)) {
    System.out.println(VEHICLES_FILE_NOT_FOUND);
    return vehicles;
}
```

Reason for accepting: the change eliminates duplication, gives the message one source of truth, and does not alter control flow, output text, or return values. The relevant repository tests continued to pass.

4. Refactoring Record 2 - Package/Path Mismatch

File	ReminderServiceTest.java
SonarQube issue	The file path should match package najah.eng.Application.service; dots should not be used in directory names.
Prompt used	Correct the test file location so it matches package najah.eng.Application.service. Do not change the package declaration or test behavior.
Decision	Accepted

Original path:
src/test/java/najah.eng.Application.service/ReminderServiceTest.java

Corrected path:
src/test/java/najah/eng/Application/service/ReminderServiceTest.java

Reason for accepting: this is a structural correction required by Java package conventions. The source content and package declaration were unchanged, so the risk was limited to file movement.

5. Refactoring Record 3 - Direct Console Logging

File	src/main/java/najah/eng/Application/Persistence/RentalRepository.java
SonarQube issue	Replace use of System.out with a logger.
Prompt used	Replace System.out calls in RentalRepository exception handlers with java.util.logging.Logger. Include the exception in the log entry and preserve all return values.
Decision	Accepted with verification tests

Original code

```
catch (IOException e) {
    System.out.println("Error saving rental.");
    return false;
}
```

Refactored code

```
import java.util.logging.Level;
import java.util.logging.Logger;

private static final Logger LOGGER =
    Logger.getLogger(RentalRepository.class.getName());

catch (IOException e) {
    LOGGER.log(Level.SEVERE, "Error saving rental.", e);
    return false;
}
```

Reason for accepting: the logger records severity and the exception stack trace, can be configured independently of normal console output, and improves maintainability. Tests used temporary directory paths to force IOException in save, read, and close operations and confirm that the repository still returned false or an empty result as designed.

6. Documentation Improvement - Complete Javadocs

Files	pom.xml and generated javaDoc/ directory
Problem	The Maven Javadoc configuration included only Vehicle.java, RentalService.java, and ReturnService.java.
Prompt used	Update the Maven Javadoc plugin so it generates documentation for all production classes and private members, without changing application behavior.
Decision	Accepted

Original configuration

```
<sourceFileIncludes>
  <sourceFileInclude>**/Domain/Vehicle.java</sourceFileInclude>
  <sourceFileInclude>**/service/RentalService.java</sourceFileInclude>
  <sourceFileInclude>**/service/ReturnService.java</sourceFileInclude>
</sourceFileIncludes>
<show>protected</show>
```

Refactored configuration

```
<!-- sourceFileIncludes removed -->
<show>private</show>
<encoding>UTF-8</encoding>
<docencoding>UTF-8</docencoding>
<charset>UTF-8</charset>
<failOnWarnings>false</failOnWarnings>
```

Reason for accepting: the revised configuration matches the requirement to generate documentation for the complete application. It changes build documentation only and does not affect runtime behavior.

7. Suggestions Rejected or Modified

7.1 Global package rename

SonarQube reported that package segments such as Application, Domain, and Persistence do not follow the lowercase package naming convention. A global rename was not performed during this phase because it would touch nearly every source file, import, test, Javadoc path, and UML reference. The team deferred this broad change to avoid unnecessary integration risk close to submission.

7.2 Behavior-changing rewrites

Suggestions that would redesign repository APIs, change text-file formats, or modify existing rental rules were not accepted. Refactoring was limited to maintainability and documentation improvements that preserved the implemented requirements.

7.3 Logging without exception context

A minimal logger call containing only the message was improved before acceptance. The final implementation passes the IOException object to `LOGGER.log` so diagnostic information is retained.

8. Verification and Human Review

- The team reviewed every proposed diff before committing it.
- The Maven tests were run after the changes.
- GitHub Actions and SonarQube Cloud analyzed pull requests.
- The selected refactoring pull request passed the Quality Gate with 0 new issues and 100% coverage on new code.
- Overall coverage later increased to 86.8%.
- The issue count decreased from 152 to 147.
- Generated Javadocs and the UML diagram were opened and inspected before submission.

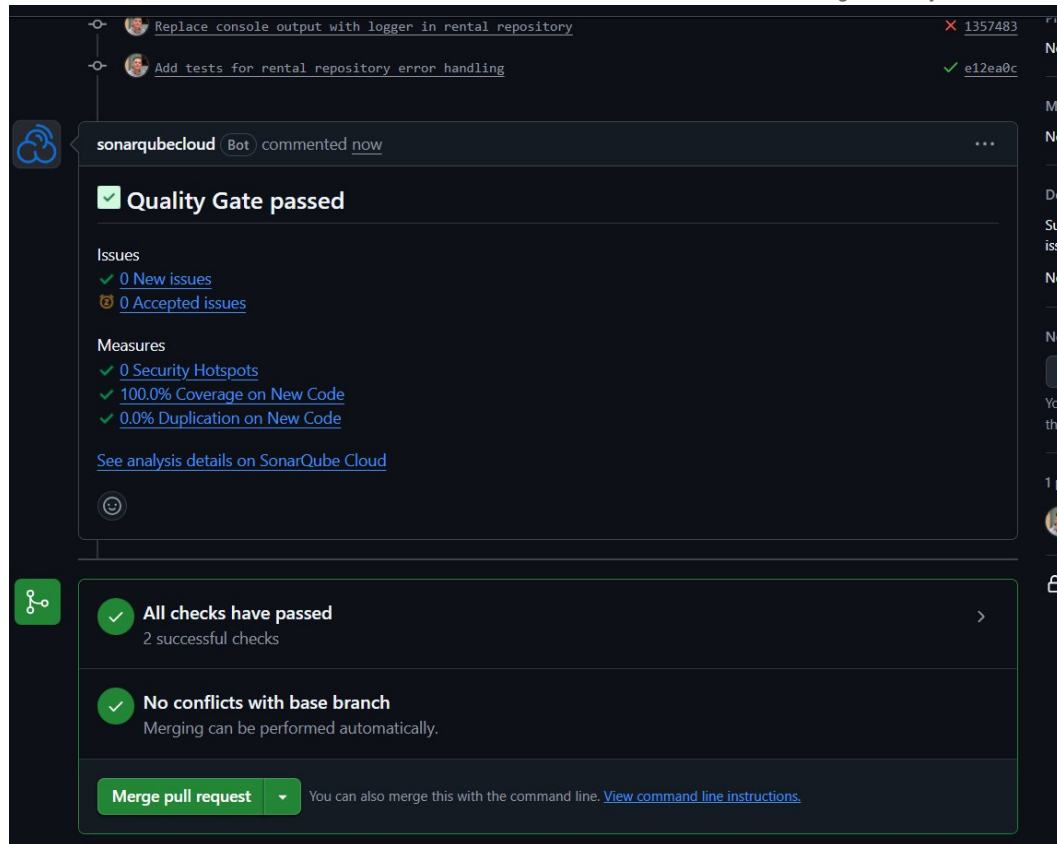


Figure 1. Automated review result after the refactoring changes.

9. AI Usage Declaration

AI was used as a refactoring and documentation assistant. The application had already been implemented by the team. AI suggestions were treated as proposals rather than authoritative changes. The team selected small changes, checked them against the existing design, reviewed the code, and relied on tests and automated analysis before acceptance.

Appendix A - Prompts

1. Refactor VehicleRepository to replace the repeated "Vehicles file not found." literal with a single constant. Preserve all current behavior and method return values.
2. Correct the ReminderServiceTest file location so it matches package najah.eng.Application.service. Do not change the package declaration or test behavior.
3. Replace System.out calls in RentalRepository exception handlers with java.util.logging.Logger. Include the exception in the log entry and preserve all return values.
4. Suggest repository tests that force IOException without modifying production behavior. The tests must verify the existing return values.
5. Update the Maven Javadoc plugin so it generates documentation for all production classes and private members.